

ATIVIDADE DA AULA 16 - Banco de Dados - Relacional (2º Semestre)

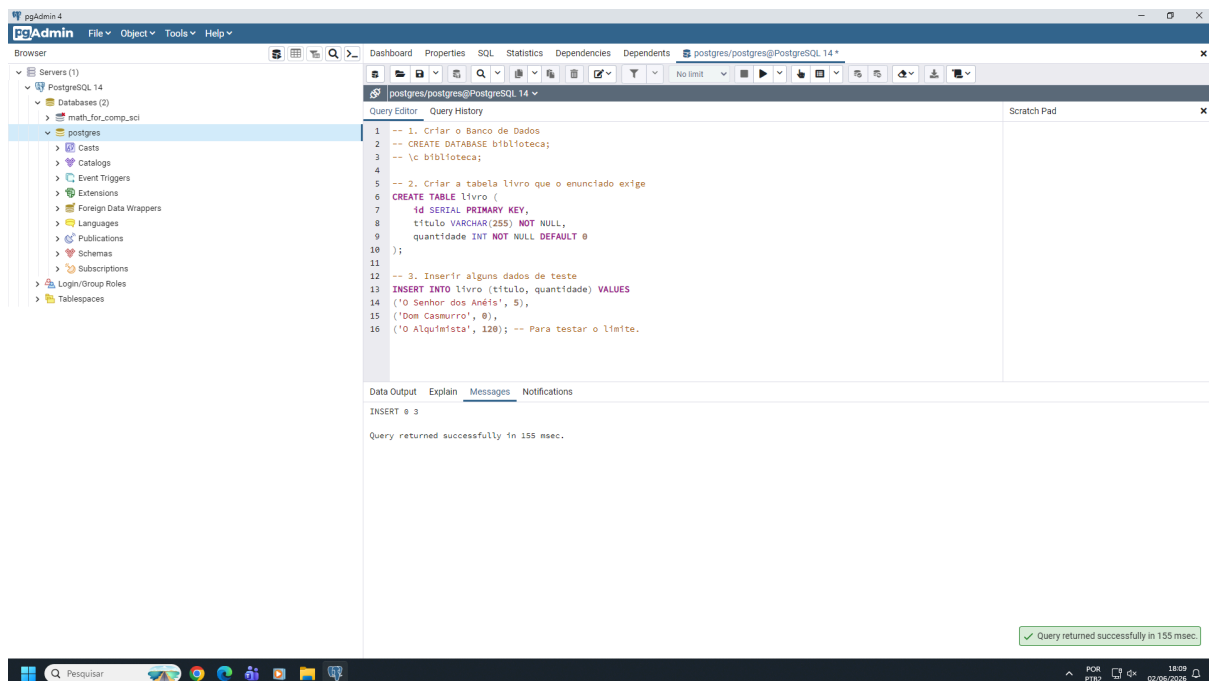
ALUNA:Nadla Fernandes Ferreira

RA:2581392613046

1º Criar banco e tabelas para fazer os exercícios. Como não foi especificado no enunciado se era para usar o mesmo banco e tabelas de um dos exercícios anteriores da disciplina de bd relacional (o db biblioteca e as tabelas dela), eu criei um banco de mesmo nome porém com dados reduzidos.

```
-- 1. Criar o Banco de Dados
-- CREATE DATABASE biblioteca;
-- \c biblioteca;

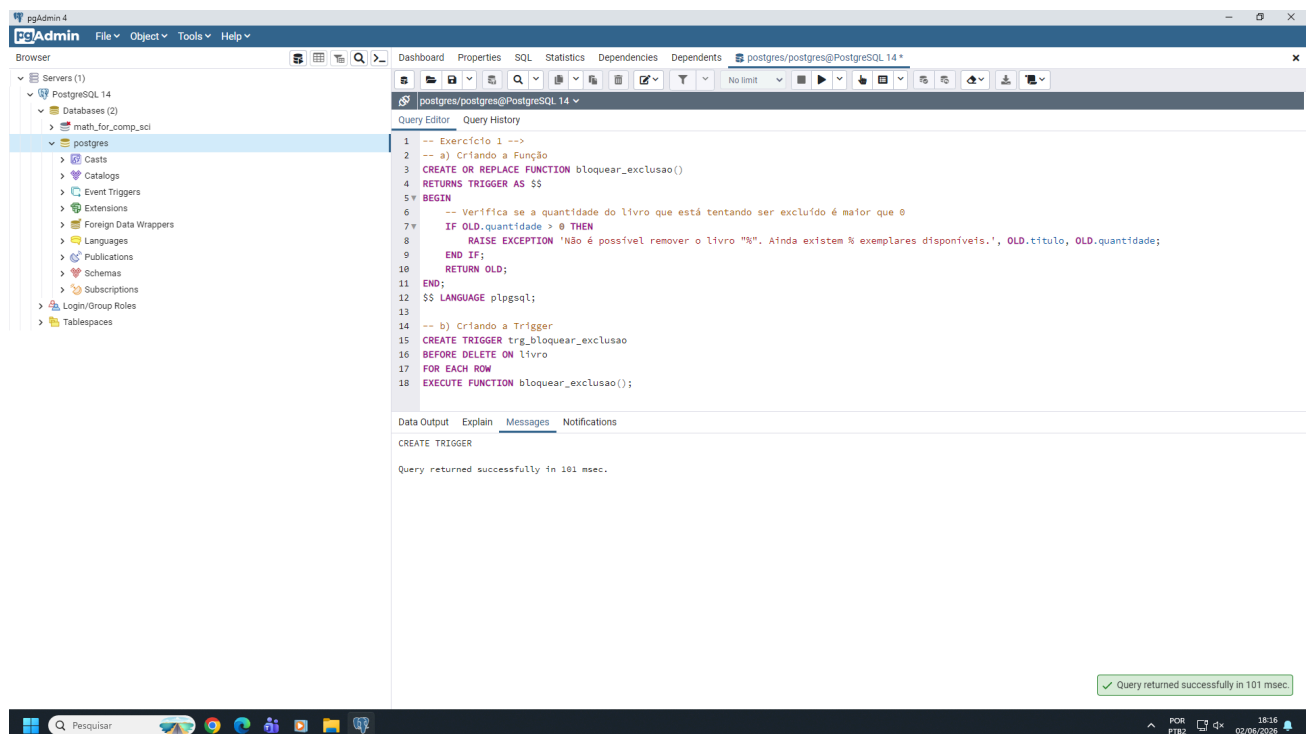
-- 2. Criar a tabela livro que o enunciado exige
CREATE TABLE livro (
    id SERIAL PRIMARY KEY,
    titulo VARCHAR(255) NOT NULL,
    quantidade INT NOT NULL DEFAULT 0
);
```



Exercício 1 →

```
-- Exercício 1 -->
-- a) Criando a Função
CREATE OR REPLACE FUNCTION bloquear_exclusao()
RETURNS TRIGGER AS $$
BEGIN
    -- Verifica se a quantidade do livro que está tentando ser
    -- excluído é maior que 0
    IF OLD.quantidade > 0 THEN
        RAISE EXCEPTION 'Não é possível remover o livro "%".
        Ainda existem % exemplares disponíveis.', OLD.titulo,
        OLD.quantidade;
    END IF;
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;

-- b) Criando a Trigger
CREATE TRIGGER trg_bloquear_exclusao
BEFORE DELETE ON livro
FOR EACH ROW
EXECUTE FUNCTION bloquear_exclusao();
```



Exercício 2 →

```
-- Exercício 2 -->
CREATE TABLE log_exclusao (
    id SERIAL PRIMARY KEY,
    titulo_livro VARCHAR(255),
    data_hora_exclusao TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    mensagem TEXT
);

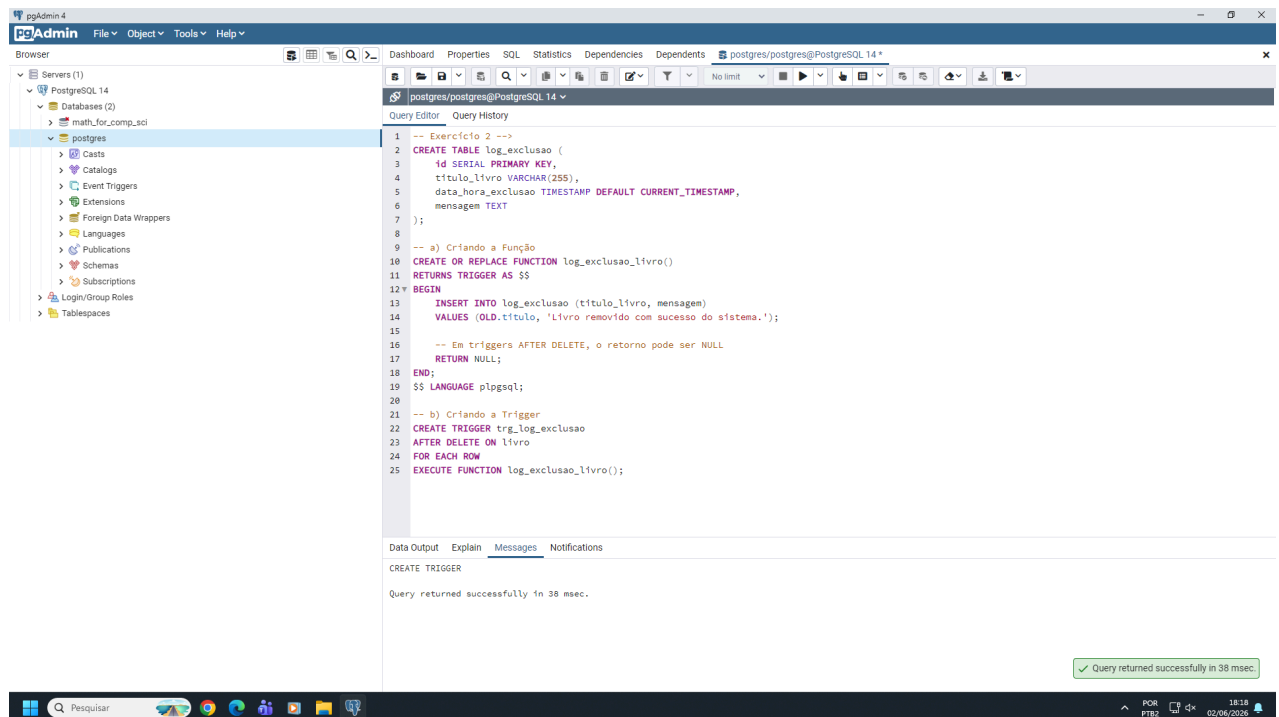
-- a) Criando a Função
CREATE OR REPLACE FUNCTION log_exclusao_livro()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO log_exclusao (titulo_livro, mensagem)
        VALUES (OLD.titulo, 'Livro removido com sucesso do
sistema.');
```

-- Em triggers AFTER DELETE, o retorno pode ser NULL

```
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;
```

-- b) Criando a Trigger

```
CREATE TRIGGER trg_log_exclusao
AFTER DELETE ON livro
FOR EACH ROW
EXECUTE FUNCTION log_exclusao_livro();
```



Exercício 3 →

-- Exercício 3 -->

-- a) Criando a Função

```
CREATE OR REPLACE FUNCTION validar_limite_estoque()
```

```
RETURNS TRIGGER AS $$
```

```
BEGIN
```

```
    -- Verifica se a nova quantidade ultrapassa 100
```

```
    IF NEW.quantidade > 100 THEN
```

```
        RAISE EXCEPTION 'Erro: A quantidade inserida (%) para  
o livro "%" excede o limite máximo permitido de 100  
unidades.', NEW.quantidade, NEW.titulo;
```

```
    END IF;
```

```
    RETURN NEW;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

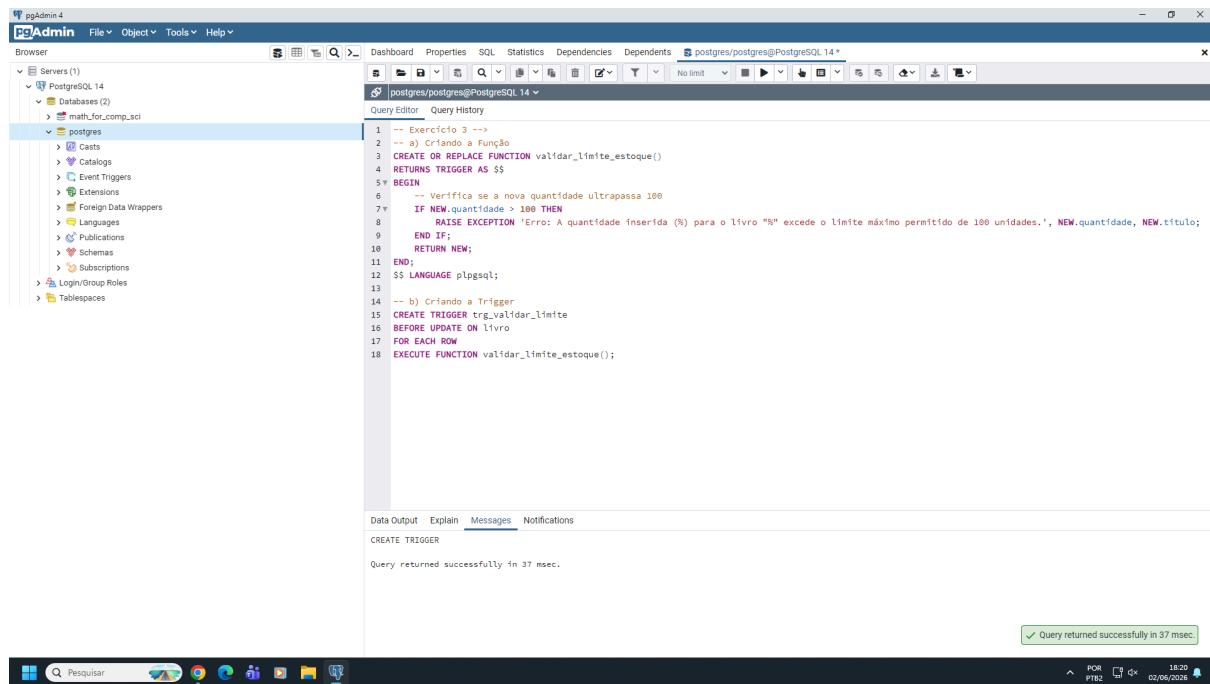
-- b) Criando a Trigger

```
CREATE TRIGGER trg_validar_limite
```

```
BEFORE UPDATE ON livro
```

```
FOR EACH ROW
```

```
EXECUTE FUNCTION validar_limite_estoque();
```



Exercício 4 →

a) A diferença entre triggers BEFORE e AFTER.

As Triggers BEFORE são executadas antes que a alteração (INSERT, UPDATE, DELETE) seja consolidada no banco de dados, permitindo validar ou modificar os dados. As Triggers AFTER rodam depois que a alteração já foi realizada com sucesso no banco.

b) Qual delas deve ser usada para validação.

Deve ser usada a trigger BEFORE. Dessa forma, se a validação falhar, a operação é interrompida antes de afetar o banco.

c) Qual delas deve ser usada para auditoria.

Deve ser usada a trigger AFTER. Pois, só faz sentido registrar algo no histórico de auditoria se a operação realmente tiver sido concluída com sucesso.

d) Por que a ordem de execução é importante em bancos de dados reais.

A ordem garante a consistência e a performance. Triggers de validação (BEFORE) evitam o processamento desnecessário de

dados inválidos. Se a ordem for invertida, o banco pode registrar logs de operações que acabaram sendo canceladas, gerando dados falsos ou inconsistentes.

Exercício 5 →

a) Quais riscos essa decisão pode gerar.

O maior risco é a inconsistência de dados. Se múltiplos sistemas usarem o mesmo banco e um deles esquecer de implementar a validação, dados corrompidos entrarão no sistema. Além disso, acessos diretos via terminal contornariam as regras.

b) Quais vantagens existem em manter regras diretamente no banco.

Centralização da regra de negócio (independente de qual aplicação acesse o banco, a regra será seguida), maior segurança de integridade e melhor performance para operações massivas de dados.

c) Explique como as triggers ajudam na integridade e consistência dos dados.

Elas agem como uma linha de defesa automatizada. Não importa como o dado chegue, a trigger intercepta a ação e garante que ela siga as regras estabelecidas.

d) Cite um exemplo real de uso de triggers em sistemas corporativos.

Sistemas bancários para registrar movimentação e de e-commerce para atualizar estoque.

Exemplo: Quando um produto é vendido, uma trigger AFTER INSERT na tabela de vendas pode atualizar automaticamente o estoque atual da tabela de produtos e gerar um registro na tabela de auditoria financeira.